

Retrieval-Augmented Generation

Connecting an LLM to Your Documents

Dr. Papa-Séga WADE

AIMS Senegal | Module 2 - Session 6

MODULE 2 - Session 6: retrieval, grounding, and document QA

"Do not guess when you can retrieve"

Where We Stopped in Session 5

Recap

Prompting reaches its ceiling on three fronts: **hallucinations**, **knowledge cutoff**, and **context window**. The model cannot answer reliably about facts it has never seen.



We are here

Baseline Objective

By the end of this session

You will understand and implement a system that lets an LLM answer questions based on **specific documents you provide**, not on what it memorized during training.

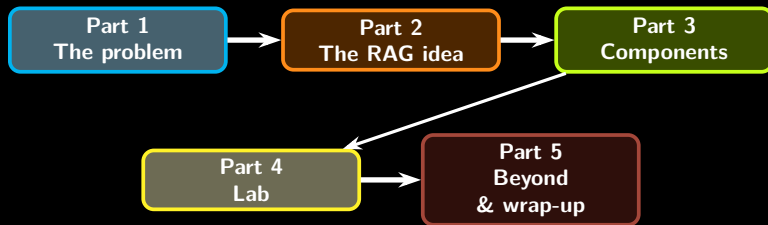
You will be able to

- explain why prompting alone cannot do this,
- build a chunks → embeddings → store → retrieve → generate pipeline,
- choose chunk sizes and embedding models,
- evaluate a retrieval against ground truth.

Guiding question

How would you build an assistant that answers students' questions about the AIMS Senegal academic regulations using **only** the official PDF?

Session Roadmap



Core message

Do not stuff knowledge into the prompt. Retrieve only what is relevant, and let the LLM synthesize.

The Problem: The LLM Does Not Know Your Documents

Concrete situation

A student asks the model:

"Quelle est la note minimale pour valider le module GAAI a AIMS Senegal cette annee?"

What the LLM does

It either invents a plausible answer (hallucination) or refuses, because the AIMS-Senegal 2025-2026 regulations were never in its training data.

Other situations of the same type

- Senegalese tax code 2025 (DGID),
- ANSD census tables for Thies,
- AIMS internal policy on lab access,
- a NGO's vaccination schedule for rural Casamance,
- a researcher's own thesis chapters.

Why Not Just Paste the Document Into the Prompt?

Three real limits

- 1 **Size**: a 300-page handbook does not fit in 8K-32K tokens.
- 2 **Cost**: even with 1M-token windows, paying for the full doc on every query is wasteful.
- 3 **Noise**: more irrelevant text means more lost-in-the-middle effect.

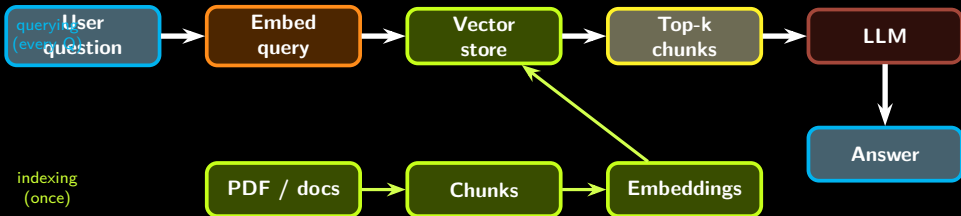
The right framing

For a student asking about **exam pass marks**, you do not need the entire AIMS regulations PDF. You need the **three paragraphs** that mention pass marks.

This is what RAG does

Find the few relevant passages, then ask the LLM to answer using only those.

The RAG Pipeline at a Glance



Two phases

Indexing happens once per document. **Querying** happens for every user question.

The Five Building Blocks

Indexing side

- 1 **Loader**: read PDF, HTML, DOCX
- 2 **Chunker**: split into passages
- 3 **Embedder**: text $\rightarrow$ vector
- 4 **Vector store**: index of vectors

Querying side

- 5 **Retriever**: similarity search
- 6 **Prompt builder**: insert top-k chunks
- 7 **LLM**: produce the answer
- 8 **(optional) Reranker**: refine top-k

Mental model

RAG = a librarian (retriever) hands the LLM the right pages, then the LLM reads and answers.

Embeddings: What They Are

Definition

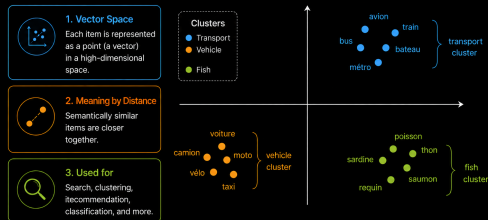
An **embedding** maps a piece of text to a dense vector in $\mathbb{R}^d$ (typically $d \in \{384, 768, 1024\}$).

Property we exploit

Texts with **similar meaning** produce vectors that are **close** in cosine distance, even if they share no words.

Embeddings: What They Are

Embeddings map items (words, sentences, images, ...) to points in a continuous vector space.



Embeddings capture meaning: items with **similar meaning** are close in the vector space, even if they are different words.

Sentence-Transformers in One Slide

What it is

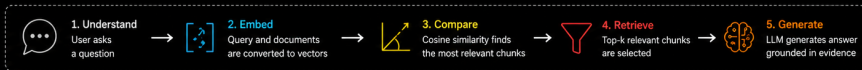
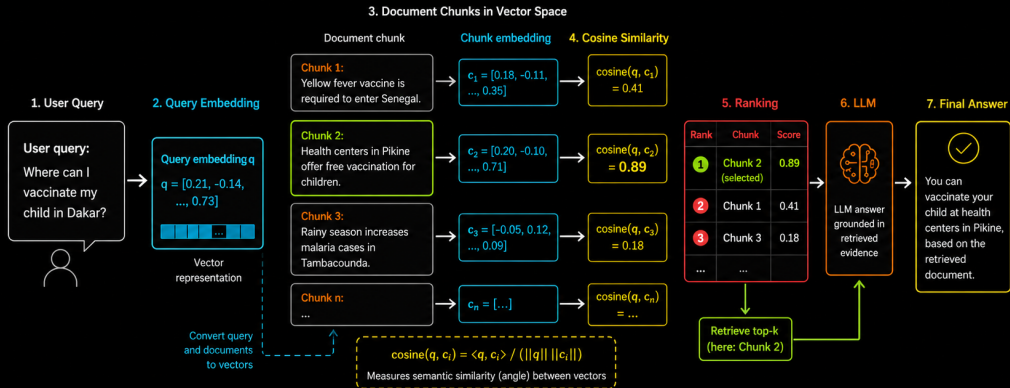
A family of pretrained encoder models (BERT-style) fine-tuned to produce **sentence-level** embeddings whose cosine distance reflects semantic similarity.

```
1 from sentence_transformers \
2     import SentenceTransformer
3
4 m = SentenceTransformer(
5     "paraphrase-multilingual-MiniLM-L12-v2"
6 )
7 v = m.encode([
8     "Comment s'inscrire a AIMS?",
9     "Procedure d'admission AIMS Senegal"
10 ])
11 # v.shape -> (2, 384)
```

Common pick for projects

- all-MiniLM-L6-v2: small, fast, English
- paraphrase-multilingual-MiniLM: French, Wolof transcribed in Latin, decent
- BGE-M3: strong multilingual, longer context

Cosine Similarity for Semantic RAG: Illustration



Concrete Example: Senegalese Health FAQ

Mini corpus (3 sentences)

- 1 D1: "Le vaccin contre la fièvre jaune est obligatoire pour entrer au Sénégal."
- 2 D2: "Les centres de santé de Pikine offrent la vaccination gratuite aux enfants."
- 3 D3: "La saison des pluies augmente les cas de paludisme dans la région de Tambacounda."

User query

"Où puis-je faire vacciner mon enfant à Dakar?"

What the embedding model does

Even though the query says "Dakar" (not "Pikine") and "enfant" matches D2, the cosine similarity will rank **D2 first**, because the meaning ("vaccinate a child in the Dakar area") is closest to D2's content.

Chunking: Why It Matters

The decision

How do we cut a long document (a 50-page AIMS handbook) into retrievable units?

Two failure modes

- chunks too small: lose context, no usable answer.
- chunks too big: dilute relevance, retrieval ranks them lower.

Practical defaults

- size: **300 to 800 tokens**
- overlap: **50 to 100 tokens** (stitches concepts split across chunks)
- respect natural boundaries: paragraphs, headings, sentences

Chunking Strategies, Compared

Strategy	How it works	When to use
Fixed-size	cut every N tokens, fixed overlap	simple, generic baseline
Recursive	split on <code>\n\n</code> , then <code>\n</code> , then sentences	narrative documents, AIMS handbooks
Semantic	cut where embedding similarity drops	long structured reports
Document-aware	respect headings, sections, table rows	regulations, laws, thesis chapters

Heuristic

Start with recursive splitting at 500 tokens / 80 overlap. Tune only if your retrieval evaluation shows a problem.

Vector Stores: Chroma in 1 Slide

What a vector store does

Stores (*vector, text, metadata*) tuples and answers **nearest-neighbor** queries fast.

When you outgrow Chroma

Production: Qdrant, Weaviate, Milvus, pgvector. Same API mental model.

Why Chroma for class

- open source, pure Python install,
- embedded mode (no separate server),
- works on Colab, on a laptop, offline,
- enough for AIMS-scale corpora.

Distance metric

Default: **cosine**. Equivalent to dot product on L_2 -normalized vectors. Fits Sentence-Transformer outputs.

Dense vs Sparse Search (Awareness)

Dense (embeddings)

- meaning-based,
- handles paraphrase and synonyms,
- weak on rare proper nouns and codes,
- example: "thiof" matches "poisson".

Sparse (BM25, TF-IDF)

- lexical overlap,
- perfect on exact codes / IDs,
- misses paraphrases,
- example: "Decret 2024-512" matches the exact passage.

Hybrid retrieval

Production systems often combine both: dense for meaning, sparse for codes and acronyms (e.g. ANSD, DGID, CFA). Mentioned for awareness; we will not implement it today.

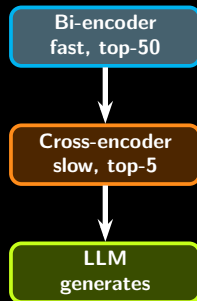
Reranking (Awareness)

Two-stage retrieval

- 1 **Bi-encoder retrieves** top-50 fast (cheap embeddings).
- 2 **Cross-encoder reranks** the 50 into top-5 (slow, accurate).

Why two stages

Cross-encoders read **query and chunk together** and score relevance. Too slow to apply to a million chunks, perfect to refine 50.



Lab Goal: Mini-RAG on an AIMS Corpus

What you will build

A working pipeline that takes a PDF (e.g. the AIMS Senegal academic regulations or a recent ANSD report) and answers questions **grounded in that document**.

Steps

- 1 Load the PDF.
- 2 Split it into chunks.
- 3 Create embeddings.
- 4 Store in Chroma.
- 5 Ask 5 questions **with** and **without** RAG.
- 6 Compare answers.

Suggested corpus options

- AIMS Senegal handbook (PDF)
- ANSD population census excerpt
- Senegalese health ministry COVID FAQ
- your own thesis chapter

Step 1-2: Load and Chunk a PDF

```
1 from pypdf import PdfReader
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3
4 # 1. Load the PDF (AIMS regulations example)
5 reader = PdfReader("aims_senegal_handbook_2025.pdf")
6 text = "\n".join(page.extract_text() or "" for page in reader.pages)
7
8 # 2. Split into chunks (500 chars, 80 overlap, paragraph-aware)
9 splitter = RecursiveCharacterTextSplitter(
10     chunk_size=500,
11     chunk_overlap=80,
12     separators=["\n\n", "\n", ". ", " ", ""],
13 )
14 chunks = splitter.split_text(text)
15 print(f"{len(chunks)} chunks generated")
```

What to observe

Open the first 3 chunks. Do they each look like coherent passages? If they cut a sentence in half, increase overlap.

Step 3-4: Embed and Store in Chroma

```
1 from sentence_transformers import SentenceTransformer
2 import chromadb
3
4 # 3. Embed all chunks
5 model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
6 embeddings = model.encode(chunks, show_progress_bar=True)
7
8 # 4. Store in Chroma (embedded mode, no server)
9 client = chromadb.PersistentClient(path="./chroma_aims")
10 collection = client.get_or_create_collection(
11     name="aims_handbook",
12     metadata={"hnsw:space": "cosine"},
13 )
14 collection.add(
15     ids=[f"chunk_{i}" for i in range(len(chunks))],
16     documents=chunks,
17     embeddings=embeddings.tolist(),
18 )
19 print(f"Indexed {collection.count()} chunks")
```

Step 5: Retrieve and Generate

```
1 from transformers import pipeline
2
3 llm = pipeline("text-generation", model="meta-llama/Llama-3.2-3B-Instruct")
4
5 def answer_with_rag(question, k=4):
6     q_vec = model.encode([question]).tolist()
7     res = collection.query(query_embeddings=q_vec, n_results=k)
8     context = "\n\n".join(res["documents"][0])
9     prompt = (
10         "You are an AIMS academic assistant. Answer using ONLY the context.\n"
11         "If the context does not contain the answer, say so.\n\n"
12         f"Context:\n{context}\n\n"
13         f"Question: {question}\nAnswer:"
14     )
15     return llm(prompt, max_new_tokens=200, do_sample=False)[0]["generated_text"]
16
17 questions = [
18     "Quel est le seuil de validation d'un module GAAI?",
19     "Combien de credits ECTS pour le semestre 2?",
20     "Quelle est la procedure d'absence justifiee?",
21 ]
22 for q in questions:
23     print(q, "->", answer_with_rag(q)[-300:])
```

Step 6: With vs Without RAG

The comparison that matters

Run the same 5 questions **without** the retrieval step (just `llm(question)`). For each, write down:

Score grid

- is the answer factually correct?
- does it cite the relevant chunk?
- does it admit "I don't know" when needed?
- is the language register right?

Expected pattern

Without RAG: confident hallucinations.

With RAG: grounded answers, occasional refusals when the corpus is silent.

Advanced Extension: Cross-Encoder Reranker

Goal

After retrieving top-20 with the bi-encoder, rerank with a cross-encoder and keep top-5. Measure if answer quality improves.

Steps

- 1 Use a cross-encoder (e.g. `ms-marco-MiniLM-L-6-v2`) or a multilingual variant.
- 2 Score (query, chunk) pairs.
- 3 Keep the 5 highest-scoring chunks.
- 4 Measure on a 20-question evaluation set: `precision@5`, answer correctness.

Expected outcome

Reranking typically lifts answer quality by 5 to 15 points, at roughly 3x retrieval latency.

Also try

Different chunk sizes (300, 500, 800) and overlaps (40, 80, 120). Measure on the same eval set.

Common Failure Modes (Be Ready)

Bad chunks

Tables and equations get mangled by naive PDF extraction. Inspect chunks before embedding.

Wrong embedder language

English-only models on French/Wolof text degrade silently. Always pick multilingual for AIMS corpora.

No-evidence answers

The LLM still hallucinates if not instructed to refuse when context is empty. Add an explicit refusal rule: "if the context is silent, answer *Je ne sais pas a partir des documents fournis*".

Stale index

When documents change, re-embed. Otherwise the system answers from the old version.

African Use Cases Where RAG Shines

Education

- AIMS academic assistant on internal regulations,
- tutor that grounds explanations in the actual course PDF,
- assistant for a Master 2 student writing a literature review.

Health

- triage assistant grounded in Ministry of Health protocols,
- rural-clinic chatbot citing official vaccination guidelines.

Public services

- DGID tax-question assistant grounded in Senegalese tax code,
- ANSD assistant for journalists querying census tables.

Agriculture

- advice agent grounded in ISRA bulletins and ANACIM weather notes,
- Wolof voice front-end + RAG over agriculture PDFs.

What to Remember

- 1 RAG = retrieve relevant chunks, then let the LLM answer with them.
- 2 Indexing happens once, querying happens every call.
- 3 Embeddings encode **meaning**, not words. Sparse search complements them.
- 4 Chunk size and overlap matter more than the choice of LLM.
- 5 Always force the model to refuse when the context is silent.
- 6 For AIMS / African contexts: pick **multilingual** embedders.

Key Resources

Core references

- RAG from Scratch Tutorial (Hugging Face Blog)
- LangChain RAG Cookbook
- Sentence-Transformers documentation
- Chroma getting started guide

Going further

- BGE-M3 multilingual embeddings (BAAI)
- Cross-encoder rerankers (MS MARCO)
- RAGAS evaluation framework (Session 8)
- LlamaIndex for production RAG

Bridge to Session 7: Fine-Tuning

Next step

RAG gives the model new **knowledge**. But what if you need it to change its **behavior**, style, or output format permanently? That is fine-tuning, with LoRA and QLoRA.



Questions & Discussion

Retrieve, then generate. Ground every answer in trusted documents.

RAG is not magic memory: it is a disciplined pipeline for search, evidence, synthesis, and verification.

Dr. Papa-Séga WADE

AIMS Senegal | GAAI-AIMS